

Lab 1 – VG

Yeganeh Vahabi

September 2026

For the VG part I looked at two things: reranking the retrieved chunks and changing the number of chunks that are returned.(limit)

The goal was to see if re-ranking can improve the results and if retrieving more or fewer chunks gives better results.

A) Reranking

setup

The baseline retrieval system selects the top three text chunks from Qdrant based on cosine similarity and inserts them directly into the prompt. To check whether this ranking was the best choice, I added a simple reranking step. Instead of taking only the top three results, the system first retrieves the top 10 candidates. Each candidate then gets a small score increase (+0.05) for every keyword from the query that appears exactly in the chunk. The chunks are reranked using this adjusted score, and the highest-ranked ones are selected.

I ran this on four test queries against the `gu.support` collection.

Results

Query: "phone number Feelgood cancellation"

Raw cosine ranking

1. Feelgood contact/phone chunk (cosine = 0.7242, 4 keyword hits)
2. "Frequently found in Contact/Help/About pages" (cosine = 0.5534, 0 hits)
3. "Applying for and being granted study support" (cosine = 0.5500, 0 hits)

Re-ranked results

1. Feelgood contact/phone chunk (combined score = 0.9242)
2. "Frequently found in Contact/Help/About pages"
3. "Applying for and being granted study support"

Reranking did not alter the ordering of the retrieved chunks. The embedding model already identified the correct answer at rank 1.

Query: "how do I cancel an appointment"

Raw cosine ranking

1. Feelgood contact/phone chunk (cosine = 0.6935, 5 keyword hits)
2. Appointment address/visiting-hours chunk (cosine = 0.5218, 3 keyword hits)
3. "Applying for and being granted study support" (cosine = 0.5209, 2 keyword hits)

Re-ranked results

1. Feelgood contact/phone chunk (combined score = 0.9435)
2. Appointment address/visiting-hours chunk
3. "Contact Student Healthcare...schedule an in-person appointment..." (combined score = 0.6696, cosine = 0.5196, 3 keyword hits)

here but re-ranking produced a modest improvement. The third-ranked result changed from a generic chunk concerning study-support applications to a chunk directly related to healthcare appointments. Although the top two positions remained unchanged, the revised ranking is more useful for answering the user's query.

Query: "emergency number"

The raw cosine retrieval already identified the correct chunk, "112 is the emergency number...", at rank 1 with a clear score advantage (0.7458 versus 0.5749 for the next result). Re-ranking did not alter the ordering.

Query: "disability support accommodations"

This query however produced the strongest evidence for the usefulness of the reranking.

Raw cosine ranking

1. "Services for people with reading disabilities on the university library website" (cosine = 0.7198, 0 keyword hits)
2. "Applying for and being granted study support" (cosine = 0.6975, 1 keyword hit)
3. "Disability study support...long-term documented disability...educational support..." (cosine = 0.6790, 2 keyword hits)

Re-ranked results

1. "Disability study support...long-term documented disability...educational support..." (combined score = 0.7790)

2. “You must have valid documentation... coordinators for study support...” (combined score = 0.7770)
3. The library-services chunk is removed from the top three results.

So cosine similarity ranked a chunk concerning library services above the chunk that actually explains the disability accommodation process. While both passages share terms such as “disability” and “support”, the first passage is less relevant to the user’s intent.

Reflection

Across four queries, reranking changed the top-3 ordering in half of them, and in one case it produced a clearly better result by removing an irrelevant chunk that outscored the correct one on pure cosine similarity. So a very cheap heuristic (like literal keyword overlap) can catch cases where semantic embedding is close enough to mislead the ranking. But it only rewards exact substring matches, so it wouldn’t catch a synonym (“appointment” vs. “booking”) or penalize an accidental keyword match in an irrelevant chunk.

B) Number of Results

Setup

The normal system returns 3 chunks. I compared this with returning only 1 chunk and returning 8 chunks. I looked at the number of characters, the sources of the chunks, and the similarity score of the last included chunk.

Results

What Is the Phone Number for Feelgood

Limit 1

The system returned 489 characters from the source `page-000-Student healthcare.txt`. The lowest similarity score was 0.6874. The answer was already present in this single chunk.

Limit 3

The system returned 1073 characters. All retrieved chunks came from the same student healthcare source, and the lowest similarity score was 0.5222.

Limit 8

The system returned 2240 characters from several different sources. The lowest similarity score dropped to 0.4693, and more unrelated information began to appear.

Increasing the number of retrieved chunks provided more context but also introduced less relevant information.

Reflection

For this example, I think a limit of 3 is a reasonable choice. It gives some extra information without adding too much unrelated text.

Using too many chunks can make the prompt larger without giving the model more useful information.